ERP Attachment Simulation Engine

Architecture, Design and Implementation Handbook

Enterprise Attachment Generation Framework for ERP Automation Testing using Java

---

VOLUME 1 — ARCHITECTURE AND FOUNDATION

Table of Contents — Volume 1

- Chapter 1: Introduction
- Chapter 2: Business Requirements
- Chapter 3: Existing Solutions Analysis
- Chapter 4: Overall Architecture
- Chapter 5: Architecture Decisions
- Chapter 6: Technology Selection
- Chapter 7: Project Structure

Chapter 1 — Introduction

1.1 Introduction

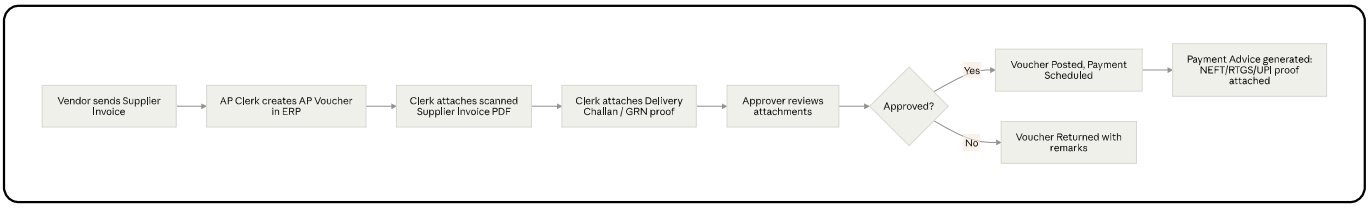
Every modern ERP system — whether it is a large-scale enterprise suite handling manufacturing and distribution, or a mid-market retail ERP like GINESYS — is built around two very different categories of documents:

1. **System-generated reports** — invoices, GRNs, purchase orders, statements, ledgers — produced *by* the ERP from its own transactional data, using its own report engine.
2. **User-uploaded attachments** — scanned or digitally created documents that a human operator attaches to a transaction as *evidence* or *supporting proof*: a scanned supplier invoice attached to an AP Voucher, a signed delivery challan attached to a GRN, a cancelled cheque image attached to a vendor onboarding form, a UPI payment screenshot attached to an advance receipt.

This handbook is concerned exclusively with the second category. It defines the architecture, design and full Java implementation of a standalone framework — the **ERP Attachment Simulation Engine** (from here on, "the Engine") — whose sole purpose is to generate large volumes of *realistic*, *deterministic*, *offline* business attachment documents (PDF, DOCX, PNG, JPG) that can be fed into ERP automation test suites wherever a human would normally attach a real-world document.

1.2 The ERP Attachment Ecosystem

To understand why this framework needs to exist, it helps to walk through where attachments actually appear inside a typical ERP transaction lifecycle. Consider a Finance module AP (Accounts Payable) Voucher flow, which is the domain this handbook's target user (a Selenium automation engineer working on GINESYS Finance automation) is most directly familiar with:



Every box in this diagram that involves an attachment (`C`, `D`, `I`) currently, in almost every ERP QA organization, depends on **one of two bad options**:

- A small, static, hand-curated folder of sample PDFs/images that testers manually seeded years ago and now reuse forever, or
- Nothing at all — the automation simply skips the attachment step, leaving an entire class of UI behavior (file upload widgets, attachment previews, attachment validation rules, virus-scan integration stubs, attachment-count business rules) permanently untested.

Both options degrade test coverage and data realism over time. Neither scales to negative testing (corrupted files, wrong file types, oversized files, documents with missing mandatory fields) or to the sheer volume needed for load and performance testing of upload-heavy flows.

1.3 ERP Reports vs. Uploaded Attachments — Why the Distinction Matters Architecturally

Dimension	ERP-Generated Report	User-Uploaded Attachment
Data source	ERP's own transactional database	External real-world entity (vendor, bank, government body)
Format control	Fully controlled by ERP's report engine (e.g. JasperReports, Crystal Reports)	Arbitrary — PDF, scanned image, DOCX, screenshot, photograph
Visual consistency	Perfectly consistent, templated	Highly variable — different vendors, different scanners, different phone cameras
Who "authors" it	The ERP	A third party outside the ERP's control
Purpose in automation	Verify report generation logic	Verify upload, storage, retrieval, validation, and display logic
Simulation approach needed	N/A (already deterministic within the ERP)	Needs a **separate simulation engine** that mimics real-world document *diversity*, not ERP report *consistency*

This distinction is the architectural seed of the entire handbook. Because uploaded attachments are inherently heterogeneous (a supplier's invoice layout looks nothing like a courier receipt, which looks nothing like a UPI screenshot), the Engine cannot be a single templating pass — it must be a **plugin-based library of independent document generators**, unified behind a common contract, each capable of producing both a "born-digital" version (clean PDF/DOCX) and a "scanned/photographed" version (JPEG/PNG with realistic imperfections) of the same logical document.

1.4 Current Problems in ERP Automation Without This Engine

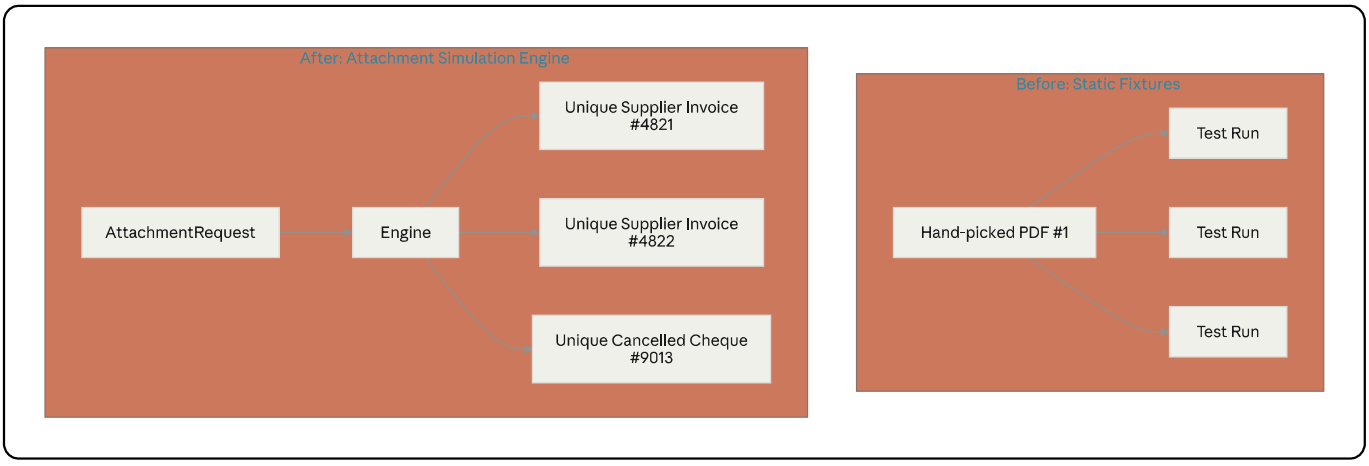
Based on direct experience automating Finance module flows (AP/AR Vouchers, GRN, PO) in frameworks such as the Selenium 4 / TestNG / Java 17 stack this handbook targets, the recurring pain points are:

1. **Static attachment pools go stale.** The same 5–10 PDFs get reused across thousands of test executions. Any business rule that depends on document *content* (e.g. "cheque date must be within 90 days," "invoice number must match PO reference") either can't be tested or requires manually editing sample files — which does not scale and is not repeatable across environments.

2. **No parameterization.** A test needing "a cancelled cheque for account ending 4521, IFSC HDFC0001234, dated today minus 3 days" has no way to get one without manual Photoshop-style editing.
3. **No negative-path coverage.** Testing "reject oversized attachment," "reject non-whitelisted file type," "reject corrupted PDF" requires artisanal, one-off broken files that nobody remembers how they were made.
4. **No volume.** Performance and load tests for attachment upload endpoints need *thousands* of unique, non-cacheable files. Static pools cannot supply this without triggering server-side dedup/caching that invalidates the test.
5. **No realism.** UI checks for "does the thumbnail preview render correctly for a rotated, slightly blurry scanned document" cannot be exercised with a pixel-perfect vector PDF exported from Word.
6. **Manual seeding breaks CI/CD.** Any dependency on a shared network drive of sample documents is fragile in Jenkins pipelines running on ephemeral agents.

1.5 Why Attachment Simulation Is Required — The Core Argument

The Engine exists to convert "attachment" from a *fixed test fixture* into a **first-class, parameterized, generatable test data type** — exactly the same conceptual shift that tools like Java Faker/Datafaker brought to *textual* test data (names, addresses, emails) a decade ago, but applied to *document-shaped* test data.



1.6 Benefits

Benefit	Description
Determinism	Given the same seed/request, the same document is reproducible byte-for-byte — critical for golden-master and visual regression testing.
Volume at speed	Designed for thousands of documents per hour on commodity hardware, using thread pools / virtual threads (Java 17+ friendly design, see Volume 6).
Offline	No network calls, no GPU, no external AI service dependency — runs on a Jenkins agent with no internet egress.
Parameterizable	Every business-relevant field (amounts, dates, GSTIN, IFSC, account numbers, vendor names) is injectable per request.
Format diversity	Same logical document can be emitted as clean PDF, clean DOCX, or scanner-simulated JPEG/PNG.
Negative testing support	First-class support for intentionally malformed, oversized, or wrong-typed output (see Volume 2, Exception & Validation chapters).
CI/CD-native	Pure JVM library, embeddable directly inside the existing `SeleniumFramework3.141_NEW`-style Maven project, no external services to provision.

1.7 Business Use Cases

1. **Functional testing** — AP Voucher creation requires attaching a Supplier Invoice and a Delivery Challan before it can be submitted for approval; the Engine supplies both, cross-referenced (same PO number, matching amounts).
2. **Negative testing** — Upload validation rejects files > 5 MB, or file types outside `{pdf, jpg, png, docx}`; the Engine can deliberately emit oversized or wrong-extension payloads.
3. **Data-driven regression** — A GRN reconciliation rule requires the attached Delivery Challan quantity to reconcile against the GRN quantity; the Engine generates matched or deliberately mismatched pairs to test both the happy path and the exception path.
4. **Performance/load testing** — An upload endpoint needs 10,000 unique 200KB–2MB files to avoid server-side caching skewing throughput numbers.
5. **UI/visual testing** — Attachment thumbnail rendering, preview pane behavior, and OCR-based auto-fill features need scanned-style images with realistic noise, rotation, and shadow — not pristine vector PDFs.

6. **Vendor/GST compliance simulation** — GST Certificate and PAN Copy attachments used in vendor onboarding flows need syntactically-valid (but obviously fictitious) GSTIN/PAN numbers matching the correct checksum/format rules, without using any real identity data.

1.8 Illustrative Examples

Attachment Type	Typical ERP Flow	Format(s) Needed
Supplier Invoice	AP Voucher creation	PDF, scanned JPG
Delivery Challan	GRN	PDF, scanned JPG
Cancelled Cheque	Vendor bank detail onboarding	PNG/JPG (scanned)
UPI Receipt	AR advance receipt	PNG (screenshot-style)
NEFT/RTGS Advice	Payment reconciliation	PDF
GST Certificate	Vendor master creation	PDF, scanned JPG
Purchase Order Copy	PO-to-GRN matching	PDF
Bank Statement	Vendor credit assessment	PDF, multi-page

1.9 Chapter Summary

The ERP Attachment Simulation Engine fills a structural gap in ERP test automation: the total absence of a scalable, deterministic, parameterizable way to produce the *human-sourced* documents that ERPs consume as attachments. Unlike ERP report generation, this problem space is characterized by format and layout *diversity*, which drives the entire architecture toward a plugin-based, strategy-driven design — introduced formally in Chapter 5 and implemented in full in Volume 2.

1.10 References

- GINESYS ERP Finance Module functional documentation (internal)
- OWASP File Upload Cheat Sheet — for negative-test rationale on file type/size validation
- ISO 8601 / RBI NEFT-RTGS format conventions — for advice-document field realism

---

Chapter 2 — Business Requirements

2.1 Introduction

This chapter formalizes what the Engine must do (functional requirements) and how well it must do it (non-functional requirements). These requirements drive every architectural

decision in Chapters 4–7 and every module boundary in Volume 2.

2.2 Functional Requirements

ID	Requirement	Priority
FR-01	The Engine shall generate at least one attachment type per category defined in Volume 4 (Invoice, Receipt, Advice, Certificate, Statement, Agreement/PO, Miscellaneous Scanned).	Must
FR-02	Every generator shall accept a strongly-typed request object exposing all business-relevant fields as optional overrides, with sensible randomized defaults when omitted.	Must
FR-03	The Engine shall support at least three output formats: PDF, PNG/JPG (raster), and DOCX, selectable independently of the document type where applicable.	Must
FR-04	The Engine shall support a "scanner simulation" post-processing pipeline (Volume 5) applicable to any raster output.	Must
FR-05	The Engine shall be seedable — an explicit seed shall produce byte-identical output across runs and machines.	Must
FR-06	The Engine shall expose a Java API (embeddable), a CLI, and an optional REST API (Volume 7).	Should
FR-07	The Engine shall support batch generation of N attachments in a single call, with independent per-item parameterization.	Must
FR-08	The Engine shall support deliberate invalid-output modes (oversized file, corrupted content, wrong extension/content-type mismatch) for negative testing.	Must
FR-09	The Engine shall allow new attachment types to be added via a plugin mechanism without modifying core framework code.	Must
FR-10	The Engine shall support localization of currency, date, and address formats (India-first, extensible).	Should
FR-11	The Engine shall integrate with JUnit/TestNG via a lightweight extension (Volume 7) so tests can request attachments inline.	Should
FR-12	The Engine shall log every generation request (type, seed, duration, output path) via SLF4J for traceability inside Jenkins build logs.	Must

2.3 Non-Functional Requirements

2.3.1 Performance

- The Engine must sustain generation of **at least 2,000 simple documents/hour** (e.g., UPI Receipt PNG) and **at least 500 complex documents/hour** (e.g., multi-page Bank Statement PDF) on a 4-core / 8GB commodity Jenkins agent.
- p99 latency for a single synchronous generation call must not exceed 400ms for raster documents and 800ms for PDF documents under single-threaded load.
- Batch APIs must scale near-linearly with available cores up to at least 8 concurrent workers (see Volume 6 benchmarking methodology).

2.3.2 Maintainability

- Each attachment type is isolated in its own module/package with no compile-time dependency on any other attachment type.
- Template content (HTML/DOCX fragments) is externalized as resources, not embedded as Java string literals, so business users/non-Java testers can tweak layout without recompiling.

2.3.3 Scalability

- The generator registry (Chapter 5, Registry Pattern) must support O(1) lookup regardless of the number of registered attachment types, so scaling to 100+ attachment types (Volume 4's full catalog) does not degrade dispatch performance.

2.3.4 Extensibility

- Adding a new attachment type must require: (1) a new request/response pair, (2) a new generator implementation, (3) a registration entry — and nothing else. No `switch` statement anywhere in core framework code should need editing (Open/Closed Principle, enforced via the Registry + Factory pattern combination in Chapter 5).

2.3.5 Security

- The Engine must never require network egress; all fonts, logos, and template resources ship inside the framework's own JARs or a configured local resource directory.
- All generated "identity-like" data (PAN, GSTIN, account numbers, IFSC codes) must be syntactically valid per public checksum/format rules but must never collide with real, in-force identifiers — achieved by using clearly-marked test ranges (e.g., PAN series `ZZZZZ` prefix conventions used by Indian tax test environments) and by documenting this explicitly (see Volume 4, GST Certificate / PAN Copy chapters).

2.3.6 Thread Safety

- All generator instances must be stateless or use `ThreadLocal`/immutable-per-call state so a single `AttachmentEngine` instance can safely serve concurrent requests from a parallel TestNG suite (`parallel="methods"` or `parallel="classes"`) without external synchronization by the caller.

2.3.7 Logging

- SLF4J is the only logging facade referenced by framework code (Chapter 6); the concrete binding (Logback, per the existing `SeleniumFramework3.141_NEW` stack using Log4j2) is left to the consuming application's classpath, per standard "library vs. application" logging hygiene.

2.3.8 Configuration

- All tunables (thread pool size, output directory, default locale, watermark toggle, scanner-effect intensity) are exposed through a single immutable `EngineConfig` object, loadable from a YAML file (SnakeYAML) or built programmatically — never from scattered system properties.

2.3.9 Internationalization

- Locale-sensitive fields (currency symbol, date format, number formatting, address block ordering) are abstracted behind a `LocaleProfile` so the initial India-first field set (INR, DD/MM/YYYY, GSTIN/PAN) can be extended later without touching generator logic.

2.4 Requirements Traceability Preview

Each functional requirement above is implemented by a specific component introduced in Chapter 4 and built in Volume 2:

Requirement	Realized By
FR-02, FR-05	`AttachmentRequest` immutable builder + `SeededRandom` (Volume 2)
FR-03, FR-04	`RendererStrategy` hierarchy + `ScannerEffectPipeline` (Volume 3, Volume 5)
FR-07	`BatchGenerationService` (Volume 2)
FR-08	`FaultInjectionPolicy` (Volume 2, Exception chapter)
FR-09	`AttachmentGeneratorRegistry` + `AttachmentGeneratorPlugin` SPI (Chapter 5, Volume 9)

2.5 Best Practices

> **Best Practice:** Treat every non-functional requirement as a *testable contract*, not an aspiration. Volume 8 defines concurrency tests and performance tests that assert directly against the numeric thresholds stated in §2.3.1.

2.6 Common Mistakes

> **Warning:** A frequent early-design mistake is conflating "randomized" with "non-deterministic." The Engine must be randomized *and* deterministic simultaneously — randomness must always flow from an explicit, capturable seed (FR-05), never from `System.currentTimeMillis()` or unseeded `Math.random()`.

2.7 Chapter Summary

The requirements above establish a framework that is functionally rich (broad document catalog, multiple formats, batch and negative-path support) while being architecturally disciplined (deterministic, thread-safe, extensible without modification). These constraints are the direct input to the architecture decisions made in Chapters 4 and 5.

Chapter 3 — Existing Solutions Analysis

3.1 Introduction

Before committing to a from-scratch Java framework, it is necessary to survey the existing landscape of document/image-generation approaches and explain, with evidence, why none of them satisfy the requirements of Chapter 2 well enough to be adopted wholesale — while acknowledging which *libraries* from that landscape are still reused as building blocks (formalized in Chapter 6).

3.2 Generative AI Image Models (Stable Diffusion, Flux)

Diffusion-based image generators can produce photorealistic document-like images from text prompts or ControlNet-guided layouts.

Advantages

- Can produce highly varied, photorealistic "scanned document" textures and camera artifacts with minimal manual effort.
- Good for pure visual noise/texture generation (e.g., paper grain, lighting variation).

Disadvantages (why unsuitable as the core engine)

- **Non-deterministic in the way required.** Even with fixed seeds, model version drift across GPU driver/library updates changes output, breaking golden-master testing (FR-05 requires *cross-machine* reproducibility, which diffusion models do not reliably guarantee without pinning an entire GPU/driver/library stack).

- **Cannot reliably render precise structured text.** Line items, exact GSTIN digits, exact totals that must reconcile with database rows are precisely what diffusion models are worst at — they hallucinate digits and characters.
- **GPU/compute dependency** violates the "runs on a Jenkins agent with no GPU" and offline constraints (§2.3.5, §1.4).
- **Throughput.** Diffusion inference at useful resolution is orders of magnitude slower than template rendering — incompatible with the 2,000 docs/hour target (§2.3.1).

Where they *are* useful: as an **offline, pre-baked asset source** — e.g., generating a fixed library of paper-texture and lighting-artifact backgrounds *once*, during framework development, which are then shipped as static resources and *deterministically composited* at runtime (see Volume 5, §5.9 texture library). This preserves realism without runtime non-determinism.

3.3 Template Engines (Thymeleaf, Freemarker, Velocity)

These are mature, deterministic, offline, high-throughput HTML/text templating engines — a strong architectural fit, evaluated in depth in Chapter 6.

3.4 PDF Libraries (OpenHTMLtoPDF, Apache PDFBox, iText)

Deterministic, offline, high-throughput PDF construction/rendering libraries — also evaluated in Chapter 6.

3.5 DOCX Libraries (docx4j, Apache POI)

Deterministic, offline DOCX generation libraries — evaluated in Chapter 6.

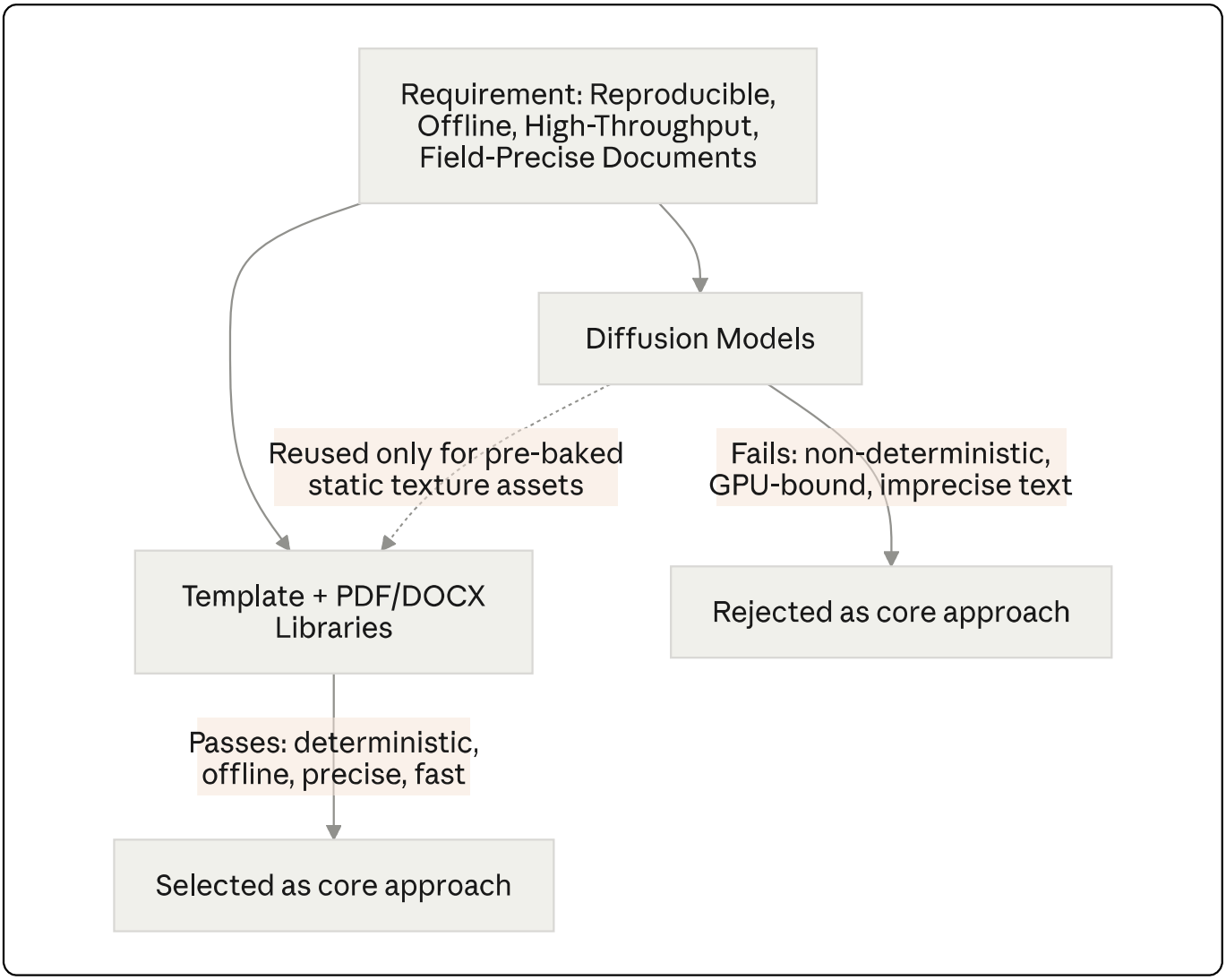
3.6 Commercial Synthetic-Document Products

Several commercial "synthetic data" vendors offer document-generation APIs (primarily targeted at training OCR/ML models rather than QA automation).

Disadvantages

- SaaS-only delivery model conflicts with the offline, no-network-egress requirement (§2.3.5) and with running inside a locked-down on-premise Jenkins environment.
- Licensing cost scales with volume, which is antithetical to a framework meant to produce *thousands* of documents per test run.
- No control over exact business-field placement needed for cross-referencing values against the ERP's own database (e.g., matching PO numbers between a generated PO Copy and the GRN screen under test).

3.7 Why Deterministic, Template-Based Generation Is Preferred



3.8 Comparison Table

Approach	Deterministic	Offline	Field-Precise	Throughput	Fit for Core Engine
Stable Diffusion / Flux	No	No (GPU)	No	Low	Not suitable (texture-asset source only)
Template Engine + PDF/DOCX libs	Yes	Yes	Yes	High	**Selected**
Commercial synthetic-doc SaaS	Partial	No	Partial	Medium	Not suitable (licensing, network)

3.9 Advantages and Disadvantages of the Selected Approach

Advantages

- Fully reproducible; supports golden-master and visual regression testing.
- Zero external dependency at runtime; safe for locked-down CI agents.
- Precise control over every business field, enabling cross-document reconciliation.
- High throughput, low resource footprint.

Disadvantages (and mitigations)

- Visual realism (scanner noise, lighting, paper texture) requires deliberate engineering effort rather than being "free," as it would be with a generative model — mitigated by the dedicated Image Processing pipeline in Volume 5, including the pre-baked texture-asset strategy from §3.2.
- Adding a genuinely new *layout family* (not just a new field set) requires a developer to build a new template, rather than just prompting a model — an accepted tradeoff, formalized as the plugin authoring workflow in Volume 9.

3.10 Chapter Summary

Generative AI image models are rejected as the *core* generation mechanism because they violate determinism, offline, field-precision, and throughput requirements — but their output is repurposed once, offline, as a static texture-asset library consumed deterministically by the Image Processing pipeline (Volume 5). The core engine is built on deterministic template + PDF/DOCX rendering libraries, selected in detail in Chapter 6.

3.11 References

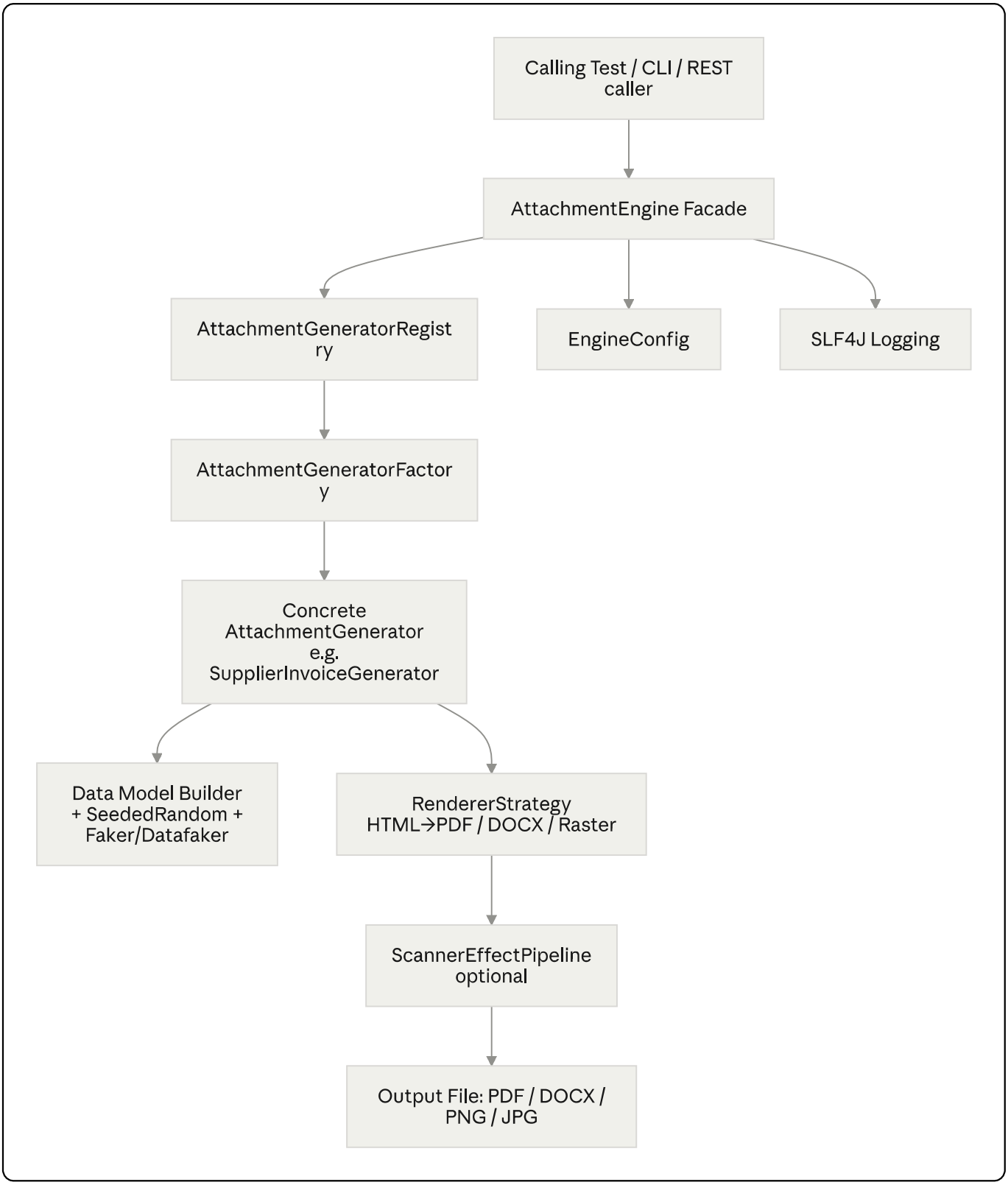
- W3C HTML-to-PDF rendering conformance notes (OpenHTMLtoPDF documentation)
- Apache PDFBox and iText licensing comparison (community documentation)

Chapter 4 — Overall Architecture

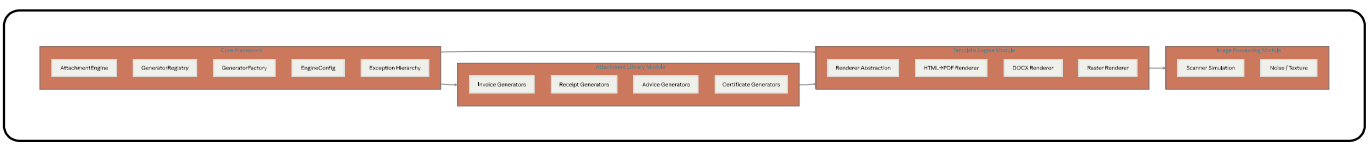
4.1 Introduction

This chapter presents the Engine's architecture at multiple levels of abstraction: high-level conceptual flow, component structure, deployment topology, request sequence, and package organization. Each diagram is deliberately drawn to make the Chapter 5 pattern choices visually obvious before they are justified in prose.

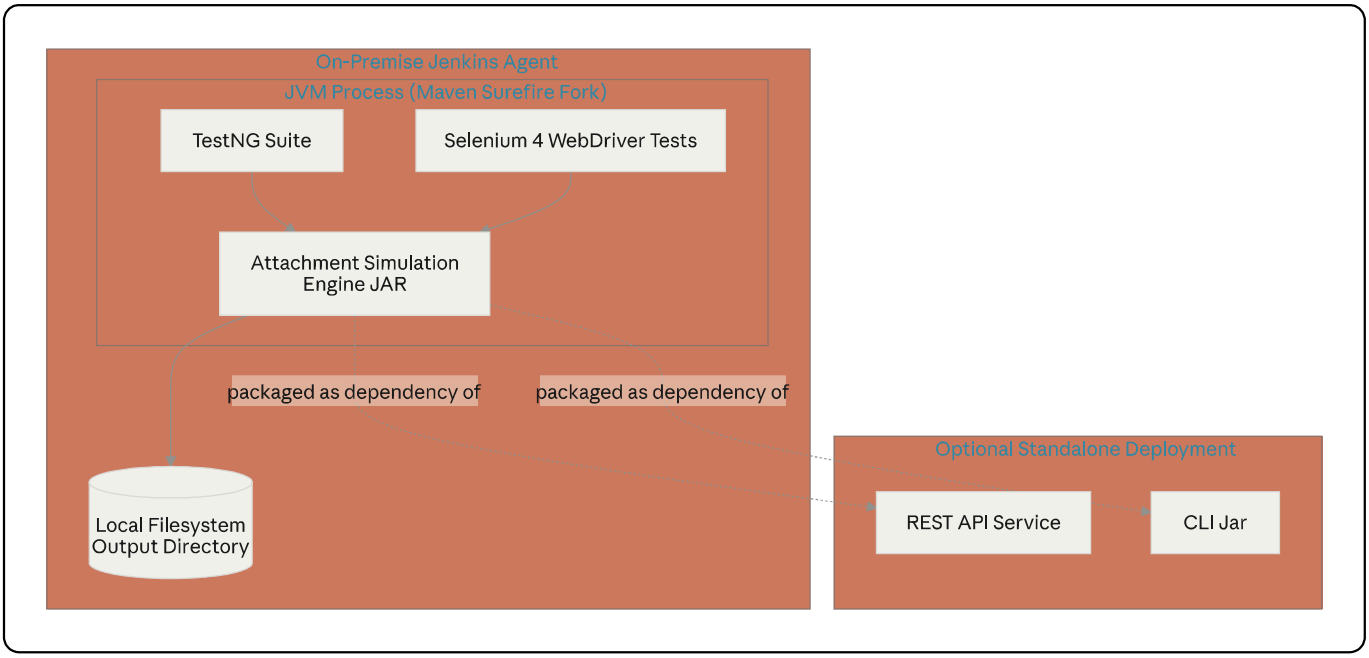
4.2 High-Level Architecture Diagram



4.3 Component Diagram

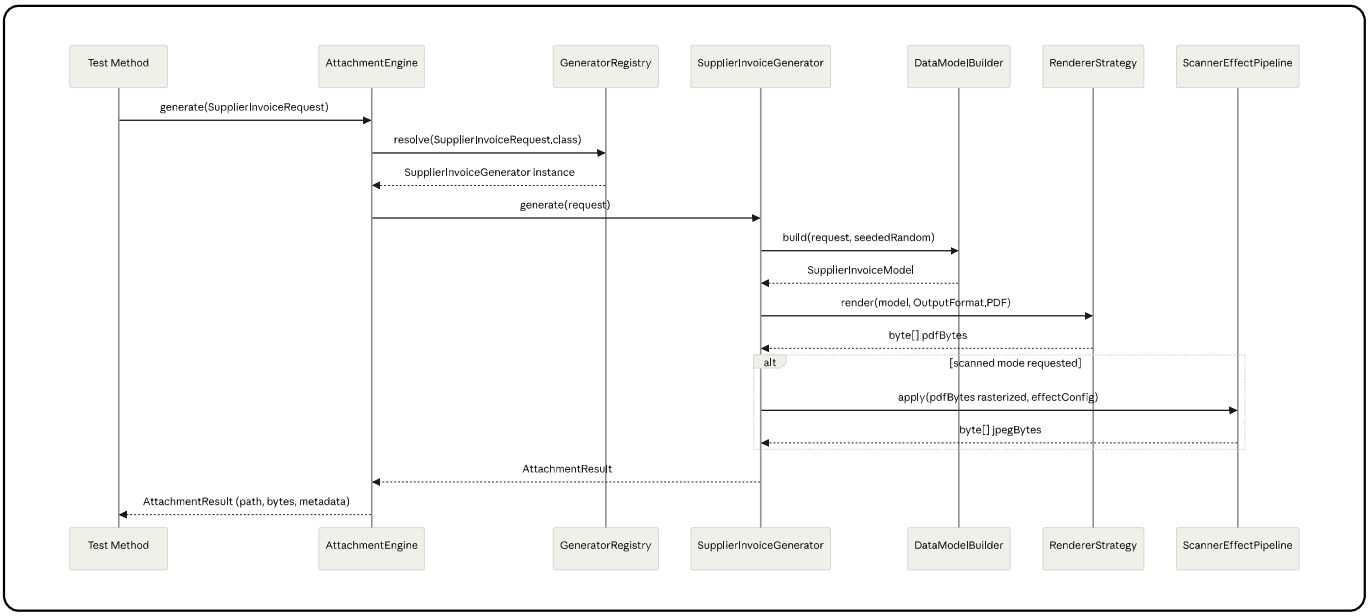


4.4 Deployment Diagram

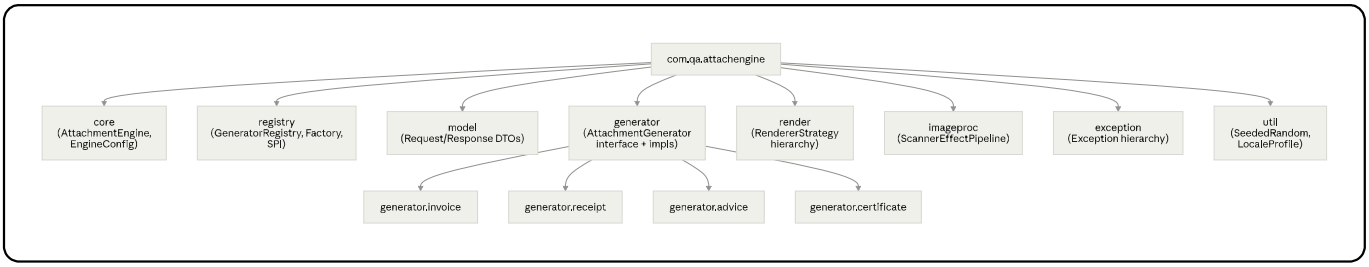


This deployment model reflects a deliberate architectural constraint: the Engine's **primary deployment form is as an embedded Maven dependency** inside the existing test framework's JVM process — not as a separately-running service. The REST API and CLI (Volume 7) are optional, secondary packaging forms of the same core JAR, never the other way around.

4.5 Sequence Diagram — Single Attachment Generation Request



4.6 Package Diagram



4.7 Design Rationale for the Layered Structure

The architecture separates four concerns that Chapter 3's analysis showed must evolve independently:

1. **What business document is this** (Attachment Library / `generator.*` packages) — grows constantly as new document types are added (Volume 4).
2. **How is it laid out and rendered** (Template Engine module) — changes rarely, shared across all document types.
3. **How realistic does the raster output need to look** (Image Processing module) — an orthogonal, optional post-processing concern.
4. **How is a request routed to the right generator** (Core + Registry) — must never change when (1) grows, per the Open/Closed Principle requirement (FR-09, §2.3.4).

This four-way separation is what makes the Chapter 5 pattern selection (Factory + Strategy + Registry + Plugin SPI) almost inevitable rather than arbitrary.

4.8 Chapter Summary

The Engine's architecture routes every request through a stable Core/Registry/Factory pipeline into an ever-growing, independently-pluggable set of Generators, each of which delegates rendering to a shared Template Engine module and, optionally, realism post-processing to the Image Processing module. This structure is deployed primarily as an embedded library inside the existing Selenium/TestNG JVM process, with CLI and REST forms as secondary packaging.

Chapter 5 — Architecture Decisions

5.1 Introduction

Chapter 4 showed *what* the structure looks like. This chapter justifies *why* each Gang-of-Four / enterprise pattern was chosen, what alternative was considered and rejected, and sketches the Java shape each decision implies (full implementations follow in Volume 2).

5.2 Factory Pattern

Decision: A `AttachmentGeneratorFactory` resolves a concrete `AttachmentGenerator<TRequest, TResponse>` implementation from a request type.

Why: Callers should depend only on the request DTO type, never on a concrete generator class name — this is what allows FR-09 (add new types without modifying core code) to hold.

```java
public interface AttachmentGeneratorFactory {
    <Q extends AttachmentRequest, R extends AttachmentResult>
    AttachmentGenerator<Q, R> resolve(Class<Q> requestType);
}
```

Alternative rejected: A single monolithic `switch` on an `AttachmentType` enum inside one giant generator class — rejected because it violates the Open/Closed Principle and turns into an unmaintainable thousand-branch method as Volume 4's ~40+ document types are added.

5.3 Strategy Pattern

Decision: Rendering (PDF vs. DOCX vs. Raster) and Scanner Effects (light noise vs. heavy degradation) are both modeled as interchangeable `Strategy` implementations.

```java
public interface RendererStrategy {
    byte[] render(DocumentModel model, RenderOptions options) throws RenderingE
}
```

Why: A generator like `SupplierInvoiceGenerator` should not know or care *how* HTML becomes a PDF versus a DOCX — that concern varies independently and must be swappable per-request (FR-03).

Alternative rejected: Format-specific subclasses of every generator (`SupplierInvoiceGeneratorPdf`, `SupplierInvoiceGeneratorDocx`) — rejected because it multiplies the class count by the number of formats, defeating maintainability (§2.3.2).

5.4 Builder Pattern

Decision: Every `AttachmentRequest` is constructed via a fluent, immutable builder.

```java
```

```
SupplierInvoiceRequest request = SupplierInvoiceRequest.builder()
    .vendorName("Acme Distributors Pvt Ltd")
    .invoiceAmount(new BigDecimal("48250.00"))
    .invoiceDate(LocalDate.now().minusDays(3))
    .seed(482910L)
    .build();
```

Why: FR-02 requires every field to be independently overridable with randomized fallback defaults; a builder with per-field `Optional`-backed setters is the cleanest way to express "override this, randomize the rest" without an explosion of constructor overloads.

5.5 Template Method

Decision: An abstract `AbstractAttachmentGenerator` defines the invariant generation skeleton (validate → build model → render → optionally post-process → wrap result → log), while concrete subclasses only supply the model-building and template-selection steps.

java

```
public abstract class AbstractAttachmentGenerator<Q extends AttachmentRequest,
        implements AttachmentGenerator<Q, R> {

    @Override
    public final R generate(Q request) {
        validate(request);
        DocumentModel model = buildModel(request);
        byte[] rendered = renderer(request).render(model, renderOptions(request
        byte[] finalBytes = maybePostProcess(rendered, request);
        return wrapResult(finalBytes, request);
    }

    protected abstract void validate(Q request);
    protected abstract DocumentModel buildModel(Q request);
    protected abstract RendererStrategy renderer(Q request);
    protected byte[] maybePostProcess(byte[] bytes, Q request) { return bytes;
    protected abstract R wrapResult(byte[] bytes, Q request);
}
```

Why: Ensures every one of Volume 4's ~40+ generators follows an identical, testable lifecycle, while still letting each one own its domain-specific logic — directly serving §2.3.2 maintainability.

5.6 Registry Pattern

Decision: A thread-safe `AttachmentGeneratorRegistry` (backed by `ConcurrentHashMap<Class<?>, AttachmentGenerator<?,?>>`) is the single place generators are registered and looked up, populated at startup via the Plugin SPI (§5.14).

Why: Satisfies §2.3.3 scalability (O(1) dispatch regardless of catalog size) and decouples "what generators exist" from "how the facade finds them."

5.7 Facade

Decision: `AttachmentEngine` is the single public entry point wrapping Registry + Factory + Config + Logging behind a small, stable API surface (`generate`, `generateBatch`, `close`).

Why: Consuming test code (and the existing `SeleniumFramework3.141_NEW` codebase) should interact with one clean class, never with Registry/Factory internals directly.

5.8 Composite

Decision: Multi-part attachments (e.g., a "Bank Statement" spanning several pages, or an AP Voucher's *set* of required attachments — invoice + challan together) are modeled as a `CompositeAttachmentResult` implementing the same `AttachmentResult` contract as a single-file result.

Why: Lets calling code treat "one document" and "a bundle of related documents" uniformly wherever the ERP flow requires attaching several files as one logical unit (§1.8 Supplier Invoice + Delivery Challan pairing).

5.9 Bridge

Decision: The abstraction "logical document" (`DocumentModel`) is bridged to its rendering implementation (`RendererStrategy`) rather than being inherited, so the same `SupplierInvoiceModel` can be rendered by an HTML→PDF renderer *or* a DOCX renderer *or* a raster renderer without three parallel class hierarchies.

Why: Directly prevents the class-explosion problem flagged as the rejected alternative in §5.3 — Bridge is the structural pattern that makes Strategy-based rendering clean at the type-hierarchy level.

5.10 Decorator

Decision: Post-processing effects (watermark, QR code overlay, scanner noise, JPEG artifacting) are applied as stackable `AttachmentPostProcessor` decorators around a base rendered byte stream.

```
java
```

```
public interface AttachmentPostProcessor {
    byte[] apply(byte[] input, PostProcessOptions options);
}
```

Why: A given document might need *any combination* of watermark + rotation + noise + compression artifacts — decoration allows arbitrary composition (`base → watermark(base) → noise(watermark(base))`) without a combinatorial explosion of classes.

5.11 Adapter

Decision: Third-party library APIs (iText's or PDFBox's native object models, docx4j's `WordprocessingMLPackage`) are never exposed directly to generator code — each is wrapped behind the framework's own `RendererStrategy` interface via a thin adapter class.

Why: Protects the framework (and Volume 6's benchmarking) from being locked into one PDF/DOCX library's API; Chapter 6's library choice can be revisited later without touching any generator.

5.12 Observer

Decision: `AttachmentEngine` optionally publishes lifecycle events (`GenerationStartedEvent`, `GenerationCompletedEvent`, `GenerationFailedEvent`) to registered `AttachmentEventListener`s.

Why: Enables Volume 6/9 concerns (metrics publishing to Grafana/InfluxDB, structured Kibana logging) to be added as *listeners*, without the core generation path knowing anything about metrics or dashboards — consistent with prior framework work integrating Grafana/Kibana observability into the existing Selenium framework.

5.13 Chain of Responsibility

Decision: Request validation (§2.6 warning on determinism, plus field-level checks like "invoice amount must be positive unless a negative-test flag is set") is implemented as a chain of `RequestValidator` handlers, each free to short-circuit with a `ValidationException` or pass to the next.

Why: New validation rules (added as Volume 4 grows) are additive — new handlers appended to the chain — never edits to one giant validation method.

5.14 Dependency Injection & Plugin System

Decision: The Engine does *not* depend on a DI framework (Spring, Guice) — per the explicit "avoid Spring Boot unless required, standalone and embeddable" constraint. Instead:

- **Dependency Injection** is achieved manually via constructor injection and a small `EngineConfig`-driven composition root (`AttachmentEngineBuilder`) — the classic

"poor man's DI" appropriate for a standalone library.

- **Plugin System** uses Java's built-in `ServiceLoader` SPI: any JAR on the classpath providing a `META-INF/services/com.qa.attachengine.spi.AttachmentGeneratorPlugin` file is auto-discovered and registered at `AttachmentEngine` startup.

```java
public interface AttachmentGeneratorPlugin {
    void register(AttachmentGeneratorRegistry registry);
}
```

Why: `ServiceLoader` is zero-dependency, JDK-native, and satisfies FR-09's "add without modifying core" requirement at the *deployment* level (a new JAR on the classpath), not just at the source-code level — the strongest possible form of extensibility, and fully consistent with "framework should be standalone and embeddable."

5.15 Summary Table of Pattern-to-Requirement Mapping

Pattern	Primary Requirement Served
Factory	FR-09 (extensibility without core edits)
Strategy + Bridge	FR-03 (multi-format output)
Builder	FR-02 (parameterized requests)
Template Method	§2.3.2 (maintainability, consistent lifecycle)
Registry	§2.3.3 (scalable O(1) dispatch)
Facade	Simplicity of the public API
Composite	§1.8 (multi-document attachment bundles)
Decorator	FR-04 (stackable scanner/watermark effects)
Adapter	Library independence (Chapter 6 reversibility)
Observer	Volume 6/9 (metrics/logging without coupling)
Chain of Responsibility	§2.6 (extensible validation)
DI (manual) + Plugin SPI	FR-09, "standalone/embeddable" constraint

5.16 Common Mistakes

> **Warning:** Do not let the Registry become a second place where format-specific `if/else` branching creeps in ("if PDF, do X; if DOCX, do Y"). That branching belongs exclusively inside `RendererStrategy` implementations (§5.3/§5.9). If you find yourself writing format checks in the Registry or Facade, it is a sign a Strategy/Bridge boundary has been violated.

5.17 Chapter Summary

Twelve patterns were deliberately selected to map one-to-one onto the requirements of Chapter 2: Factory/Registry/Plugin-SPI jointly deliver extensibility without core modification; Strategy/Bridge/Adapter jointly deliver multi-format, library-independent rendering; Builder delivers parameterization; Template Method and Chain of Responsibility deliver consistent, extensible lifecycle and validation; Composite and Decorator deliver compositional flexibility for bundled documents and stacked visual effects; Observer decouples observability. None were chosen for pattern's sake — each closes a specific requirement gap identified in Chapter 2.

Chapter 6 — Technology Selection

6.1 Introduction

This chapter benchmarks and selects the concrete third-party libraries that sit behind the abstractions defined in Chapter 5 (`RendererStrategy`, `AttachmentPostProcessor`, etc.). The guiding principle, per §5.11 (Adapter), is that **every selection below is swappable** — this chapter documents the *initial* choice and its rationale, not a permanent lock-in.

6.2 Template Engines: Thymeleaf vs. Freemarker vs. Velocity

Criterion	Thymeleaf	Freemarker	Velocity
Natural-templating (valid HTML as template)	Yes	No	No
Standalone use (no servlet container)	Yes	Yes	Yes
Maturity / maintenance activity	Active	Active	Largely dormant
Learning curve for HTML-based invoice layouts	Low (designers can preview raw HTML)	Medium	Medium
Integration with HTML→PDF renderers (Ch. 6.3)	Excellent (produces clean HTML/CSS)	Good	Good

Selection: Thymeleaf. Its "natural templating" property — a template *is* valid, previewable HTML — is decisive for Volume 3's requirement that invoice/receipt layouts be designed and reviewed as ordinary HTML/CSS before being fed to the PDF renderer, and it pairs cleanly with OpenHTMLtoPDF's CSS-driven rendering model (§6.3).

6.3 PDF Libraries: OpenHTMLtoPDF vs. Apache PDFBox vs. iText

Criterion	OpenHTMLtoPDF	Apache PDFBox	iText
Input model	HTML/CSS → PDF	Low-level PDF object construction	Low-level PDF object construction + HTML add-on (commercial)
Licensing	LGPL/MIT-family, free	Apache 2.0, free	AGPL / commercial dual license
Fit for template-driven invoices	Excellent	Poor (manual coordinate-based drawing)	Good, but licensing cost at scale
Fit for low-level tasks (watermarks, metadata, merging)	Poor	Excellent	Excellent

Selection: OpenHTMLtoPDF as the primary renderer (consuming Thymeleaf output directly), **with Apache PDFBox as a secondary, low-level utility library** for post-render operations that OpenHTMLtoPDF cannot do well: watermark overlay, metadata injection, page merging for multi-page Bank Statements, and digital-signature-style visual stamps (Volume 3). iText is explicitly avoided as a core dependency due to its AGPL/commercial licensing model, which is a poor fit for a framework meant to be freely embedded across an organization's test estate.

6.4 DOCX Libraries: docx4j vs. Apache POI

Criterion	docx4j	Apache POI
DOCX-specific ergonomics	Purpose-built for WordprocessingML	General-purpose (also handles XLSX/PPTX)
Template-based mail-merge style generation	Strong native support	Requires more manual XML manipulation
Existing framework familiarity	New	Already used elsewhere in Office-format tooling generally

Selection: docx4j for DOCX-format Vendor Agreement / Approval Letter generation (Volume 4), because its variable-replacement and content-control APIs map directly onto the Builder-pattern request objects from §5.4. Apache POI remains available as a fallback adapter (§5.11) if a future attachment type needs XLSX (e.g., a "Bank Statement" delivered as a downloadable spreadsheet rather than PDF).

6.5 Barcode/QR: ZXing vs. Barcode4J

Criterion	ZXing	Barcode4J
QR code support	Excellent, actively maintained	Limited
1D barcode support (for courier/transport receipts)	Good	Excellent

Selection: ZXing for QR codes (UPI receipts, GST certificate verification QR) **and Barcode4J for 1D barcodes** (courier/transport receipt tracking numbers) — a deliberate split rather than forcing one library to cover both formats poorly.

6.6 Test Data Generation: Java Faker vs. Datafaker

Criterion	Java Faker	Datafaker
Maintenance status	Unmaintained since ~2020	Actively maintained fork
Locale/provider coverage	Frozen	Growing
Java 17 module-system friendliness	Untested	Verified

Selection: Datafaker. As the actively-maintained successor to the now-dormant Java Faker, it is the only defensible choice for a framework built on Java 17 with a multi-year maintenance horizon (§2.3.2).

6.7 Image Processing: Apache Batik vs. TwelveMonkeys vs. ImageIO

Criterion	Apache Batik	TwelveMonkeys	Java ImageIO (JDK built-in)
SVG rendering (logos, signature vector art)	Excellent (this is its purpose)	No	No
Extended raster format support (TIFF, extra JPEG variants)	No	Excellent	Baseline only
Zero extra dependency	No	No	Yes (JDK-included)

Selection: All three, layered. Java's built-in `ImageIO` handles baseline PNG/JPEG read/write with zero dependency; **TwelveMonkeys** plugs into `ImageIO`'s own SPI to extend format support (needed for some scanner-artifact source textures, §3.2); **Apache Batik** is used specifically for rendering vector logo/signature assets to raster at generation time, since letterhead logos are most maintainably authored as SVG.

6.8 Data Interchange: Jackson vs. SnakeYAML

Selection: Both, for different purposes. Jackson (with `jackson-databind`) handles JSON serialization of `AttachmentRequest`/`AttachmentResult` objects for the REST API and CLI (Volume 7). SnakeYAML is used exclusively for `EngineConfig` file loading (§2.3.8), since YAML's human-editability is specifically valuable for a config file that testers/CI engineers will hand-edit, whereas request/response payloads exchanged programmatically are better served by JSON's stricter, more tool-friendly schema.

6.9 Logging: SLF4J + Logback

Selection: SLF4J as the sole compile-time logging dependency (§2.3.7), with **Logback recommended, but not mandated, as the runtime binding** in standalone/CLI/REST deployments. When embedded inside `SeleniumFramework3.141_NEW`, the Engine's SLF4J calls transparently flow into that project's existing Log4j2 binding — no double-logging-framework conflict, because the framework never depends on a concrete binding itself.

6.10 Testing: JUnit vs. Mockito

Selection: JUnit 5 (Jupiter) + Mockito, matching the de facto standard for modern Java 17 library testing and directly supporting Volume 8's unit/integration/concurrency test chapters. Note this is independent of the *consuming* framework's TestNG choice — the Engine's own internal test suite uses JUnit 5, while consumers are free to invoke the Engine from TestNG-based Selenium suites, since the Engine exposes a plain Java API with no test-framework coupling.

6.11 Final Technology Stack Summary

Concern	Selected Library
HTML templating	Thymeleaf
HTML→PDF rendering	OpenHTMLtoPDF
Low-level PDF manipulation	Apache PDFBox
DOCX generation	docx4j
QR codes	ZXing
1D barcodes	Barcode4J
Fake/randomized data	Datafaker
SVG→raster (logos/signatures)	Apache Batik
Extended raster format support	TwelveMonkeys (via ImageIO SPI)
JSON serialization	Jackson
YAML config	SnakeYAML
Logging facade	SLF4J (binding-agnostic)
Internal unit testing	JUnit 5 + Mockito

6.12 Best Practices

> **Best Practice:** Every one of the above libraries is consumed exclusively through the Adapter boundary defined in §5.11. No generator class in Volume 4 imports `org.apache.pdfbox.*` or `org.docx4j.*` directly — only `com.qa.attachengine.render.*` interfaces.

6.13 Common Mistakes

> **Warning:** A common integration mistake is binding a concrete SLF4J implementation (e.g., `logback-classic`) inside the Engine's own `pom.xml` as a non-`test`-scoped dependency. This forces a specific logging backend onto every consumer and is exactly the "double logging framework" conflict §6.9 is designed to avoid. Logback must remain `<scope>test</scope>` only within the Engine's own build.

6.14 Chapter Summary

Every technology selection in this chapter traces back to a Chapter 2 requirement or a Chapter 3 rejection, and every library sits behind a Chapter 5 abstraction boundary, making the entire stack replaceable component-by-component as better alternatives emerge — a direct, practical consequence of the Adapter and Strategy/Bridge patterns chosen earlier.

Chapter 7 — Project Structure

7.1 Introduction

This chapter defines the physical Maven multi-module layout that hosts the logical package structure from Chapter 4 (§4.6) and the technology stack from Chapter 6. This structure is what Volume 2 onward will populate with actual source files.

7.2 Maven Multi-Module Overview

```
graph TD
    Parent["attachment-simulation-engine (pom, parent)"]
    Parent --> Core["engine-core"]
    Parent --> Render["engine-render"]
    Parent --> ImageProc["engine-imageproc"]
    Parent --> Library["engine-attachment-library"]
    Parent --> Cli["engine-cli"]
    Parent --> Rest["engine-rest-api"]
    Parent --> Bom["engine-bom (dependency management)"]
```

7.3 Module Responsibilities

Module	Responsibility	Volume Where Built
`engine-core`	`AttachmentEngine` facade, Registry, Factory, Config, exception hierarchy, request/response base types	Volume 2
`engine-render`	`RendererStrategy` implementations (Thymeleaf/OpenHTMLtoPDF/docx4j adapters)	Volume 3
`engine-imageproc`	Scanner simulation, noise, watermarking decorators	Volume 5
`engine-attachment-library`	All concrete generators (Invoice, Receipt, Advice, Certificate, …) — depends on `engine-core` + `engine-render`	Volume 4
`engine-cli`	Command-line packaging (`picocli`-based)	Volume 7
`engine-rest-api`	Optional REST API packaging (plain `com.sun.net.httpserver` or minimal servlet, no Spring Boot per constraint)	Volume 7
`engine-bom`	Centralized dependency version management (`<dependencyManagement>`) consumed by all other modules	Volume 2

7.4 Top-Level Folder Structure

```
attachment-simulation-engine/
├── pom.xml                                  (parent POM)
├── engine-bom/
│   └── pom.xml
├── engine-core/
│   ├── pom.xml
│   └── src/main/java/com/qa/attachengine/
│       ├── core/
│       ├── registry/
│       ├── model/
│       ├── exception/
│       └── util/
├── engine-render/
│   ├── pom.xml
│   └── src/main/java/com/qa/attachengine/render/
│       ├── html/
│       ├── docx/
│       └── raster/
├── engine-imageproc/
│   ├── pom.xml
│   └── src/main/java/com/qa/attachengine/imageproc/
├── engine-attachment-library/
│   ├── pom.xml
│   └── src/main/java/com/qa/attachengine/generator/
│       ├── invoice/
│       ├── receipt/
│       ├── advice/
│       └── certificate/
│   └── src/main/resources/templates/
│       ├── invoice/
│       ├── receipt/
│       └── ...
├── engine-cli/
│   ├── pom.xml
│   └── src/main/java/com/qa/attachengine/cli/
├── engine-rest-api/
│   ├── pom.xml
│   └── src/main/java/com/qa/attachengine/rest/
└── docs/
    └── handbook/    (this handbook's markdown volumes)
```

7.5 Naming Conventions

Element	Convention	Example
Module artifact IDs	`engine-<concern>`	`engine-render`
Base package	`com.qa.attachengine.<module-concern>`	`com.qa.attachengine.generator.invoice`
Request DTOs	`<DocumentName>Request`	`SupplierInvoiceRequest`
Result DTOs	`<DocumentName>Result`	`SupplierInvoiceResult`
Generators	`<DocumentName>Generator`	`SupplierInvoiceGenerator`
Renderer adapters	`<Library><Format>Renderer`	`OpenHtmlToPdfRenderer`
Templates (Thymeleaf)	`<document-name>.html`, kebab-case	`supplier-invoice.html`
Exceptions	`<Concern>Exception`, extending `AttachmentEngineException`	`RenderingException`, `ValidationException`

7.6 Packaging Standards

- Every module publishes a `-sources.jar` and `-javadoc.jar` (via `maven-source-plugin` / `maven-javadoc-plugin`) so `engine-attachment-library` and consuming projects like `SeleniumFramework3.141_NEW` get full IDE-level documentation without needing this handbook open side-by-side.
- `engine-bom` pins every third-party library version from Chapter 6 exactly once; no other module declares an explicit `<version>` for a Chapter 6 dependency — all versions flow from the BOM's `<dependencyManagement>` block, preventing version drift between, e.g., `engine-render`'s OpenHTMLtoPDF version and `engine-imageproc`'s TwelveMonkeys-compatible ImageIO expectations.
- `engine-attachment-library`'s Thymeleaf template resources are packaged under `src/main/resources/templates/` so they are classpath-loadable both from the JAR (production/CI use) and directly from the filesystem during template-authoring/debugging (a Thymeleaf `ClassLoaderTemplateResolver` vs. `FileTemplateResolver` toggle exposed via `EngineConfig`, per §2.3.8).

7.7 Example Parent POM Skeleton

```xml
<project xmlns="http://maven.apache.org/POM/4.0.0">
    <modelVersion>4.0.0</modelVersion>
    <groupId>com.qa.attachengine</groupId>
    <artifactId>attachment-simulation-engine</artifactId>
    <version>1.0.0-SNAPSHOT</version>
    <packaging>pom</packaging>

    <properties>
        <maven.compiler.source>17</maven.compiler.source>
        <maven.compiler.target>17</maven.compiler.target>
        <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
    </properties>

    <modules>
        <module>engine-bom</module>
        <module>engine-core</module>
        <module>engine-render</module>
        <module>engine-imageproc</module>
        <module>engine-attachment-library</module>
        <module>engine-cli</module>
        <module>engine-rest-api</module>
    </modules>
</project>
```

7.8 Best Practices

> **Best Practice:** Keep `engine-core` free of any Chapter 6 third-party rendering/image dependency. It should compile against interfaces only (`RendererStrategy`, `AttachmentPostProcessor`). This is what makes the Adapter pattern (§5.11) enforceable at the *build* level, not just by convention — `engine-core`'s `pom.xml` simply has no `<dependency>` entry through which a violation could occur.

7.9 Common Mistakes

> **Warning:** Do not let `engine-attachment-library` depend on `engine-cli` or `engine-rest-api` "for convenience" (e.g., to reuse a CLI argument-parsing helper). Dependencies must only ever point inward toward `engine-core`/`engine-render`/`engine-imageproc`, never outward toward packaging modules — otherwise the module graph in §7.2 becomes cyclic and unbuildable in the intended dependency order.

7.10 Chapter Summary

The physical Maven layout mirrors the logical architecture from Chapter 4 almost exactly: one module per architectural concern, dependencies flowing strictly inward from packaging modules (CLI/REST) through the attachment library into the core/render/imageproc foundation, with a BOM module ensuring the Chapter 6 technology stack's versions never drift apart across modules.

7.11 References

- Apache Maven multi-module reactor build documentation
- Maven `dependencyManagement`/BOM pattern (Apache Maven official guide)

End of Volume 1. Volume 2 ("Core Framework") builds the full Java implementation of every interface and pattern introduced in this volume — `AttachmentEngine`, the Registry/Factory, request/response object hierarchy, exception hierarchy, and thread-safe caching/resource-management layer.